

bcrypt

Seminario #01 - Algoritmos de Criptografia - Tema 09

Dupla: Isabela Sousa e Paulo Victor Almeida

Entrega: 06 de outubro (codigo + video no Classroom ate as 19h)

1. Regras do trabalho (resumo do enunciado)

- Valor: 7,5 pontos. Apresentacao = 50% da nota; apresentacao + respostas as perguntas = os outros 50%.
- Entregar no Classroom: codigo-fonte (qualquer linguagem) + video curto explicando o projeto, ate a hora da apresentacao (19h).
- Dupla que nao entregar video e codigo perde 1,0 ponto. Dupla que nao apresentar perde 100% da nota.
- A apresentacao deve ser feita pelos DOIS membros da dupla.
- Dicas do grupo: NAO usar biblioteca pronta; nomes de metodos em portugues (facilita responder perguntas); descrever os passos do algoritmo; o professor corrige antes da apresentacao quando o codigo e enviado antes.

2. O que e o bcrypt

bcrypt e uma funcao de hash de SENHAS (nao um hash de uso geral). Foi criada por Niels Provos e David Mazieres em 1999 (USENIX), para o OpenBSD, e e baseada na cifra de bloco Blowfish (Bruce Schneier, 1993).

Ideia central: em vez de ser rapido como SHA-256 ou MD5, o bcrypt e PROPOSITAMENTE LENTO e ajustavel. Isso torna ataques de forza bruta e de dicionario muito caros. Ele tambem embute um SAL aleatorio no proprio hash, o que impede rainbow tables e faz senhas iguais gerarem hashes diferentes.

Tres propriedades-chave

- Sal de 128 bits (16 bytes) aleatorio por senha.
- Fator de custo (work factor): 2^{custo} iteracoes. Cada +1 dobra o tempo. Hoje se recomenda custo 12 ou mais.
- Adaptavel: com o passar dos anos e o aumento do poder de hardware, basta aumentar o custo.

3. Por que nao usar SHA-256/MD5 para senhas?

SHA-256 e MD5 foram feitos para serem rapidos: uma GPU calcula bilhoes de hashes por segundo, entao um atacante testa bilhoes de senhas por segundo. O bcrypt, com custo 12, limita isso a poucas dezenas de tentativas por segundo por nucleo. E ainda usa 4 KB de memoria (as caixas S), o que dificulta GPUs/ASICs (mas nao tanto quanto scrypt/Argon2 - Tema 10).

4. Base: a cifra Blowfish

- Bloco de 64 bits (duas metades de 32 bits), rede de Feistel com 16 rodadas.
- Vetor P com 18 subchaves de 32 bits + 4 caixas S de 256 palavras (4 KB) dependentes da chave.
- Valores iniciais de P e S = digitos hexadecimais de PI ($P[0] = 0x243F6A88$). Prova de que nao ha backdoor escondida.
- Funcao $F(x) = ((S0[a] + S1[b]) \text{ XOR } S2[c]) + S3[d]$, onde a,b,c,d sao os 4 bytes de x (somadas mod 2^{32}).
- Rodada: $E = E \text{ XOR } P[i]$; $D = D \text{ XOR } F(E)$; troca E e D. Ao final: $D \hat{=} P[16]$, $E \hat{=} P[17]$.

Ponto forte do Blowfish para o bcrypt: a preparacao da chave (key schedule) e cara - reescreve P e as 4 caixas S cifrando blocos repetidamente (521 cifragens). O bcrypt explora e repete isso muitas vezes.

5. O algoritmo passo a passo (Eksblowfish)

Eksblowfish = 'Expensive Key Schedule Blowfish'. Entradas: senha (ate 72 bytes), sal (16 bytes), custo.

Passo	O que acontece	No codigo
1	Senha -> UTF-8 + byte \0, corta em 72 bytes	preparar_senha
2	Estado <- digitos de PI (P e S)	EstadoBlowfish
3	ExpandKey(estados, sal, senha)	expandir_chave(...,sal)
4	Repete 2^custo vezes:	expansao_custosa
4a	ExpandKey(estados, 0, senha)	expandir_chave
4b	ExpandKey(estados, 0, sal)	expandir_chave
5	Cifra 64x 'OrpheanBeholderScryDoubt' (ECB)	calcular_hash_bruto
6	Monta \$2b\$custo\$sal(22)hash(31) em base64	gerar_hash

ExpandKey(estados, sal, chave) em detalhe

- $P[i] = P[i] \text{ XOR (palavras da chave, repetida ciclicamente) para } i = 0..17$.
- Bloco = 0. Se houver sal: XOR do bloco com as palavras do sal (alternando as 4 palavras).
- Cifra o bloco com o estado atual; o resultado substitui $P[0], P[1]$; repete para $P[2..17]$.
- Continua do mesmo jeito para as 4 caixas S ($S[0..255]$, S1, S2, S3), sempre cifrando o resultado anterior.
- Com sal = 0, e o key schedule normal do Blowfish. Alternar chave e sal nas iteracoes faz o sal influenciar todo o processo.

Formato do hash de 60 caracteres

```
$2b$12$R9h/cIPz0gi.URNNX3kh20PST9/PgBkqquzi.Ss7KIUg02t0jWMUW
|  |  |_____ 22 chars _____|_____ 31 chars _____|
|  |      sal (16 bytes)           hash (23 bytes)
|  +-- custo: 2^12 = 4096 iteracoes
+-- versao ($2b$ = versao corrigida)
```

Alfabeto base64 do bcrypt: ./A-Za-z0-9 (diferente do base64 padrao). Verificacao de senha: le custo e sal do hash guardado, recalcula com a senha digitada e compara (em tempo constante, para evitar ataque de temporizacao).

6. Limitacoes e pontos de atencao

- Limite de 72 bytes: o que passa disso na senha e ignorado. Solucao comum: pre-hash (ex.: SHA-256 + base64) antes do bcrypt.
- Senha nao pode conter byte nulo (\0 e o terminador).
- Usa pouca memoria (~4 KB): melhor contra GPU que SHA, mas pior que scrypt/Argon2 (memory-hard).
- Versoes: \$2a\$ (original, com bug de tamanho), \$2x\$/\$2y\$ (correcoes), \$2b\$ (atual).
- Alternativas modernas: Argon2id (recomendado pela OWASP), scrypt, PBKDF2.

7. Nossa implementacao (mapa para responder perguntas)

Arquivo	Funcoes principais
constantes.py	calcular_pi_fracionario, gerar_tabelas_iniciais
blowfish.py	EstadoBlowfish, funcao_f, cifrar_bloco, expandir_chave,

	expansao_custosa
bcrypt.py	preparar_senha, calcular_hash_bruto, gerar_hash, verificar_senha, base64
main.py	Menu: gerar, verificar, custo x tempo, efeito do sal
testes.py	6 hashes identicos aos da lib oficial (vazia, acentos, >72 bytes)

Sem bibliotecas: nem o hash nem as tabelas do Blowfish vem de fora - os digitos de PI sao calculados no proprio codigo (formula de Machin com inteiros grandes) e conferem com os valores oficiais. A biblioteca 'bcrypt' e usada APENAS em testes.py como gabarito.

Como rodar: `python main.py` | `python testes.py` (requer pip install bcrypt so para o teste).
Repositorio: github.com/viquituor/seminario-bcrypt

8. Roteiro sugerido de apresentacao (~10 a 12 min)

Divisao sugerida entre os dois membros - ambos DEVEM falar e ambos devem saber responder tudo.

Tempo	Quem	Conteudo
0:00-1:30	Isabela	O que e senha hash, por que SHA/MD5 nao servem, historia do bcrypt
1:30-3:30	Isabela	Sal, custo (2^n), formato do hash de 60 chars
3:30-6:00	Paulo	Blowfish (Feistel, P e S, digitos de PI) e Eksblowfish
6:00-8:00	Paulo	Passo a passo do algoritmo (tabela da secao 5)
8:00-10:30	Isabela	Demo ao vivo: main.py opcoes 1, 4, 3 (e testes.py)
10:30-12:00	Ambos	Limitacoes (72 bytes), Argon2/scrypt, conclusao

Demo ao vivo (sugestao)

- Opcao 1: gerar hash de 'senha123' com custo 4 mostrando os passos [1]..[4] na tela.
- Opcao 4: mesma senha duas vezes gera hashes diferentes (efeito do sal).
- Opcao 3: tabela custo x tempo - cada +1 dobra o tempo.
- Opcao 2: verificar senha correta e incorreta.
- `python testes.py`: mostrar que os hashes sao IDENTICOS aos da biblioteca oficial.
- Dica: use custo baixo (4-6) na demo; Python puro e lento (custo 8 ~ 6 s).

9. Roteiro do video (2 a 3 min)

- Apresentar-se e o tema (bcrypt, para que serve).
- Mostrar a estrutura das pastas e explicar em 3 frases o que cada arquivo faz.
- Rodar main.py: gerar um hash com passo a passo e verificar a senha.
- Rodar testes.py mostrando que bate com a biblioteca oficial.
- Encerrar com a limitacao dos 72 bytes e o Argon2 como alternativa.

10. Perguntas provaveis e respostas curtas

P: Por que o bcrypt e lento de proposito?

R: Para encarecer a forca bruta. O custo (2^n iteracoes de ExpandKey) e ajustavel conforme o hardware evolui.

P: Para que serve o sal?

R: Evita rainbow tables e faz senhas iguais terem hashes diferentes. Fica guardado no proprio hash (nao e segredo).

P: O que acontece se aumentar o custo em 1?

R: O tempo dobra (o dobro de iteracoes).

P: Por que 'OrpheanBeholderScryDoubt'?

R: Texto fixo de 24 bytes (3 blocos de 64 bits), cifrado 64 vezes com o estado gerado pela senha. O texto em si é arbitrário (escolhido pelos autores); o importante é ser um valor fixo e conhecido.

P: Bcrypt e criptografia ou hash?

R: É uma função de hash de senhas (KDF) construída sobre uma cifra. Não é reversível: não existe 'descriptografar'.

P: Por que o limite de 72 bytes?

R: O vetor P tem 18 palavras x 4 bytes = 72 bytes; além disso a chave só repetiria/ignoraria dados.

P: Como verifica a senha?

R: Extrai custo e sal do hash guardado, recalcula com a senha digitada e compara em tempo constante.

P: De onde vem os valores de P e S?

R: Dos dígitos hexadecimais de PI. No nosso código são calculados por aritmética de inteiros grandes (fórmula de Machin).

P: Qual a diferença entre bcrypt e Argon2?

R: Argon2 (vencedor da PHC 2015) também é configurável em memória e paralelismo (memory-hard), resistindo melhor a GPU/ASIC.

P: Pode usar \$2a\$, \$2b\$, \$2y\$?

R: São versões; \$2b\$ é a atual. Nossa verificação aceita 2a/2b/2y.

P: Qual custo usar hoje?

R: Ao menos 12 (ajustar para ~250 ms a 1 s por hash no servidor).

P: Por que a implementação em Python é mais lenta?

R: Python interpretado faz cada cifração em laço; C leva milissegundos. Por isso a demo usa custo baixo.

11. Checklist de entrega

- [] Enviar código (pasta/zip ou link do GitHub) no tópico do Seminário #01 no Classroom.
- [] Enviar o vídeo (2-3 min) no mesmo tópico - até às 19h do dia da apresentação.
- [] Ensaiar a demo (custo baixo) e ter o hash de exemplo pronto.
- [] Os dois membros treinam as perguntas da seção 10.
- [] Se possível, mandar o código antes para o professor dar retorno.

Referências

- Provos, N.; Mazieres, D. A Future-Adaptable Password Scheme. USENIX Annual Technical Conference, 1999.
- Schneier, B. Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish). 1994.
- Vídeo indicado: youtube.com/watch?v=A6mh3-HvY0k (DES - criptografia simétrica, base de cifras de Feistel).
- Site indicado: page.math.tu-berlin.de/~kant/teaching/hess/krypto-ws2006/des.htm (DES ilustrado).
- OWASP Password Storage Cheat Sheet.